



算法自学进阶路线

蓝桥杯定制 · 一轮基础版

核心目标：从零打牢蓝桥杯竞赛入门到中阶算法地基，完成第一轮系统铺垫。不涉及ACM高级内容（网络流、AC自动机、主席树等）。第二轮将在本路线基础上进行专题深挖与难度升级。

▢ **数学底线：**高中数学（集合、函数、对数、排列组合、数列）完全够用。离散数学、概率论等不需要系统学，学到对应算法时按需补一两个概念即可，切莫本末倒置。



刷题方法与学习习惯（贯穿全程的铁律）

在开始之前，先把这些原则刻进习惯里。它们比任何算法都更能保护你不跑偏。

1. **一道题最多想30分钟**，想不出来直接看题解。不要死磕一道题浪费一天，这是自学者最容易犯的错误。
2. **看完题解必须独立写一遍**，不复制粘贴。写完后对比题解代码，找自己哪里写得不好。

3. **建立错题本**：做错的、不会的、有启发的题都记录下来。每周复盘一次，重点看当时是思路错了还是代码写错了。
4. **质量 > 数量**：吃透100题远胜囫囵吞枣1000题。基础模板题要达到肌肉记忆——闭着眼睛能写出来。
5. **自己构造测试用例**：写完代码不要直接提交，先自己跑几个简单用例和边界用例，这能帮你提前发现大部分bug。
6. **一个专题一个专题学**：不同时开多个专题，但学新专题时主动串联旧知识——队列→BFS、并查集→Kruskal、回溯→记忆化搜索→DP、Prim→Dijkstra。
7. **知识串联大于死记硬背**：不按厚重的算法书顺序死磕，按知识关联性递进。知识没有孤岛，每一环都是上一环的自然应用。
8. **增加模拟赛**：每个阶段结束后安排1次洛谷入门赛模拟，全部阶段结束后每周1次，培养综合解题能力和时间感。
9. **先暴破再优化**：DP或难题没思路，立刻退回回溯/递归写暴力版本（哪怕超时）。在暴力版本中找重叠子问题，自然就会想到优化方向。

时间预估指南（一轮基础·紧凑版）

重要前置说明：以下时间是为“打基础”设定的紧凑区间。这一轮不要求做难题、不求一次吃透所有变种，只要能识别算法类型、

套用基础模板、理解核心原理，就算达标。第二轮会针对薄弱点做专项强化。

不同投入节奏下的总耗时参考：

- **每天1-2小时**（课程繁忙、在职学习）：约 **5-7个月**
- **每天2-3小时**（有固定课余时间的大学生）：约 **4-5个月**
- **每天3-4小时**（假期全职冲刺、备赛集训）：约 **3-4个月**

如果你的进度比预估慢一个月，完全正常。但如果某一阶段远超上限，先自查是否在纠结超纲难题——一轮学习要敢于“先标记、后回来”。

 各阶段时间分配一览

阶段	内容	定位	题量	预计时间 (紧凑)
一	C语言+模拟	建立代码翻译能力	10-15题	3-5周
二	转C++	掌握STL，效率翻倍	重写阶段一	2-3周
三	效率工具包	基础算法+复杂度	20-25题	2-4周
四	链表(指针实现)	线性结构收官	6-8题	2-3周

阶段	内容	定位	题量	预计时间 (紧凑)
五	树+堆 +DFS+回溯	递归思维总 决战	20-25题	6-8周
六 A	图论	模板化，战 术喘息	15-20题	4-6周
六 B	系统贪心	图论延续 →DP引子	12-15题	1-3周
七	动态规划	蓝桥杯核心 战场(一轮)	15-20题	3-5周
合 计			约120- 140题	约5-8个 月

一轮定位：完成即胜利。本轮结束后，你已经拥有了完整的算法知识图谱和120+题的实战经验。第二轮再对薄弱模块（通常是DP、贪心、搜索）进行专题深挖，那时你的理解速度和解题能力将完全不同。

第一阶段：跨越新手村（C语言与思维构建）

预计时间 约3-5周

学什么：C语言基础。重点搞定数组、字符串、结构体、函数四大块。指针的底层概念需要了解（变量地址、取地址符 `&`、解引用符 `*`），但不需要用指针动态分配内存（不碰 `malloc/free`）。这一阶段的目标是理解“变量在内存中长什么样”，为后面“用指针操作链表和树”埋下伏笔。

怎么学：语法过关后，直接去力扣找10-15道“模拟题”开刷。

如果C语言的编译错误多到让你怀疑人生，可以直接跳到第二阶段用C++起步，回头再看C的指针概念即可。这条路不是宗教，不用殉道。

为什么这么学：数组和字符串最常用，结构体帮你建立“复合数据类型”的概念。刷模拟题让你适应计算机的思考方式——把“人脑的思路”精准翻译成“代码逻辑”，这是所有算法能力的底层地基。刷着刷着你会碰到超时(TLE)或内存超限——先别慌，说明你开始意识到“写出来”和“写好”是两回事。为什么会超时、怎么评估代码效率，放到第三阶段结束后再系统学，那时候你有对比，感受深得多。

重点练习边界条件处理：数组越界、空值判断、字符串长度处理，这是90% bug的来源。

学会用printf调试：不要依赖IDE断点调试，先练会用printf打印变量值定位问题，这是竞赛最实用的调试法。

配套题目（基础模拟）： LC9, LC7, LC263, LC66, LC58, LC14, LC13, LC28, LC67, LC27

检验标准：能独立写出任意一道模拟题，代码没有语法错误，能处理常见边界条件。

第二阶段：鸟枪换炮（转战C++）

预计时间 约2-3周

学什么：开始用C++。

为什么这么学：前面你已经用C语言磕磕绊绊写过模拟题了，现在切到C++，你会真正发出一声感叹：“STL库是真他娘的好用！”有对比才知道优劣。

你只需要学C++的这一个极小子集，其他一概不管：

1. 基本语法与C几乎一样：变量、循环、条件判断、函数
2. 输入输出：cin/cout + 必须写的优化代码：

```
ios::sync_with_stdio(false);  
cin.tie(nullptr);
```

3. STL容器：vector（动态数组）、string（字符串）、queue（队列）、stack（栈）、**deque（双端队列）**、priority_queue（优先队列/堆）、**set/unordered_set（集合，偶尔用于去重与快速查找）**、unordered_map（哈希表）
4. STL算法：sort（排序）、swap（交换）、reverse（反转）、next_permutation（全排列）

5. C++11实用特性：auto、范围for循环

注：deque、set/unordered_set 暂不深入，知道有这些工具，碰到相应场景时能想起来即可。

绝对不要学：面向对象（类、继承、多态）、模板、异常处理、智能指针。这些对算法竞赛毫无帮助。

过渡方法：把第一阶段的所有C代码，用C++和STL重写一遍。

检验标准：能用C++和STL重写第一阶段所有代码；输入输出不会超时。

第三阶段：效率启蒙（基础算法工具）

预计时间 约2-4周（紧凑推进，不钻难题）

这一阶段的目标不是成为某个算法的高手，而是认全“工具箱里有哪些螺丝刀”。看懂每个工具解决什么问题、什么时候用它，能写出基础题的代码即可。后续阶段会反复使用这些工具，用多了自然熟练。

学什么（按顺序）：栈与队列 → 双指针 → 排序 → 二分查找 → 哈希表 → 前缀和与差分 → 滑动窗口 → 数学基础

怎么学：栈和队列死记“先进后出”和“先进先出”，用STL练手；带着双指针的思想学排序；二分查找体会“数组有序时的查找速度”；前缀和/差分搞定区间操作；哈希表掌握“空间换时间”的思维；数学重点搞懂质数筛、GCD、快速幂、排列组合。

学完这阶段后补一件事：系统学时间复杂度和空间复杂度。此时你已经亲手写过双指针、二分、排序，一对比马上理解为什么 $O(\log n)$ 比 $O(n)$ 快。以后看到数据量，能直接反推代码最多套几层循环。

你现在写的所有东西——栈、队列、哈希表——底层全是数组。
下一个阶段，我们要用指针来做链表。

重点拆解：

- **双指针：**对撞指针（依赖排序）、快慢指针（不依赖排序）、滑动窗口（双指针特例）
- **排序：**手写冒泡/选择/插入理解原理和稳定性，STL的sort解决实际问题
- **二分查找：**核心认知——二分的本质是“二段性”，而非“有序性”。一件事能转化为“是/否”判断且“前面全是，后面全否”，就能二分。进阶用二分答案。**先按“有序性”理解，做5道基础二分；等做到LC153时，再回来看“二段性”这句话，会瞬间悟道。**
- **哈希表：**把 $O(n)$ 查找降到 $O(1)$ ，遇“查找/计数/去重”第一反应用哈希表
- **前缀和与差分：**面试竞赛极高频，掌握快速求和与区间修改
- **数学基础：**质数筛、GCD、快速幂、排列组合，各练2-3题
- **复杂度速查：**
 - $n \leq 20$ ： $O(2^n)$

- $n \leq 100$: $O(n^3)$
- $n \leq 1000$: $O(n^2)$
- $n \leq 10^5$: $O(n \log n)$
- $n \leq 10^6$: $O(n)$

反向推导训练：看到数据范围，立刻反推算法天花板。这是竞赛中“看题猜算法”的核心能力。

- $n \leq 10^6 \rightarrow$ 大概率 $O(n)$ ，不能套超过一层的循环
- $n \leq 10^5 \rightarrow O(n \log n)$ 是上限，想到排序、堆、二分
- $n \leq 2000 \rightarrow O(n^2)$ 可接受，可以考虑简单DP的二重循环
- $n \leq 300 \rightarrow O(n^3)$ 可接受，Floyd、区间DP
- $n \leq 20 \rightarrow O(2^n)$ 回溯、状态压缩

配套题目：

- 栈与队列：LC20, LC232, LC225
- 二分查找/二分答案：洛谷 P2678, P1873, P1182；力扣 LC34, LC35, LC69, LC74, LC167
- 哈希表：力扣 LC1, LC217, LC242, LC349
- 前缀和与差分：洛谷 P1147, P8218, P2367, P3406
- 滑动窗口：力扣 LC209, LC594, LC643, LC3
- 数学基础：洛谷 P3383（质数筛），P1029（GCD），P1226（快速幂）

 **套壳题（锻炼读题抽象能力）：**LC14（字符串遍历）、洛谷 P2678（二分答案）——题目背景很长，但底层算法就是基础工具。

检验标准：看到数据量能说出最多套几层循环；熟练运用双指针、二分、哈希表、前缀和。

第四阶段：夯实基本功（线性数据结构·指针实现）

预计时间 约2-3周

学什么：链表（指针实现法）。

怎么学与程度：用 C++ 的 `struct` 和指针来实现链表。力扣上所有链表题都是基于 `ListNode` 结构体，直接用指针操作原生接口，方便验证和调试。重点搞懂链表的增删改查，以及它和数组的本质区别——数组连续内存、支持随机访问但插入删除慢；链表离散、插入删除快但不能随机访问。这个区别在树和图的结构中会反复用到。

经典题型：反转链表、快慢指针找中点/环、合并有序链表、删除倒数第 n 个节点。反复写3遍以上。学会用虚拟头节点处理边界，能大幅简化代码。

不需要手写 `new` 和 `delete` 的内存管理——力扣题目中节点已由系统创建并传入，你只管操作指针的指向。真正需要自己创建节点的情况（如合并链表），用 `new ListNode(val)` 即可。

配套题目：LC21, LC83, LC141, LC203, LC206, LC876

 **套壳题：**LC141（环形链表）——用快慢指针，故事背景可以包装成各种场景。

检验标准：能闭眼写出链表经典题；理解指针的赋值、断链、重连操作。

第五阶段：核心过渡（树、搜索与回溯）

预计时间 约6-8周。这是全路线最烧脑的阶段，允许自己慢下来，每一道题都值得花时间想透。

学什么（按顺序）：堆与优先队列 → 二叉树（指针实现） → DFS：万能钥匙 → 回溯算法

5.1 堆与优先队列

先花点时间理解堆的原理。**用数组手写一遍小顶堆的插入和删除（push和pop），各写一次即可。**这一遍的价值在于：以后看到priority_queue的源码不会觉得是黑魔法，堆排序、TopK、Dijkstra优化三处都会感谢这次手写。手写完之后，永远用STL的priority_queue。

这是解决“动态最值”问题的神器——TopK、数据流中位数。更重要的是，图论阶段Dijkstra必须用它。

5.2 二叉树（指针实现法）

直接用 C++ 的 `struct TreeNode` 和指针来实现，与力扣接口一致。用 `left`、`right` 指针表示左右孩子。

前中后序遍历刷10-15题。弄清楚为什么这题用前序、那题用后序——**理解把每个节点都当根节点处理的“分治思维”**。必须在纸上画递归树，这步跳不了。递归和迭代两种写法都要掌握——迭代让你亲手管栈，看清“执行栈”和“回溯”全过程。

层序遍历与 BFS：用第三阶段学的队列实现，体会按层扩散。此时已经串联：**队列 → BFS → 层序遍历**。

数组模拟法（用下标代替指针）了解一下即可，竞赛中图的孩子表示法会用到这种思想，但二叉树本身直接用指针更直观、更好调试。

5.3 DFS：万能钥匙（回溯之前先建立这个认知）

在正式学回溯之前，先理解 DFS 的战略地位。DFS 不只是一个图遍历算法，它是竞赛中最强的“兜底武器”：

1. **所有 DP 题都可以先用 DFS 写暴力版**：想不出状态转移方程？先用 DFS 枚举所有可能，然后找重叠子问题，加上 memo 就是记忆化搜索。
2. **所有没思路的题都可以先写 DFS**：哪怕会超时，先写出来确保逻辑正确，再想办法优化。有代码在手，比对着白纸空想强一百万倍。
3. **DFS 的思维模型非常简单**：每一步枚举所有可能的选择，无法前进就回溯。这个模型可以套在任何“求方案数、求最值、求路径”的问题上。

第五阶段的回溯算法，本质就是“带剪枝的结构化 DFS”。先把 DFS 的底子打牢，回溯只是加上“提前判断这条路值不值得走”这一步。

5.4 回溯算法

先搞定**全排列、子集、组合、分割**四个基础题型，各练2-3题。刚开始看不懂题解正常，手动推导一个简单例子。能独立默写模板、搞懂“选择→递归→撤销”，就算合格。N皇后选做，不适合入门死磕。

关键动作——剪枝：练会“在枝桠入口判断是否直接返回”，这是区分暴力枚举和高效回溯的分水岭。

回溯遇到大数据会超时，原因是重复计算太多。先记住“回溯 → 超时 → 重复计算”这条线，第七阶段的记忆化搜索会给你解药。

配套题目：

- 二叉树：LC144, LC145, LC94, LC100, LC101, LC102, LC104, LC110, LC111, LC112
- 回溯：LC17, LC22, LC39, LC46, LC77, LC78, LC131（N皇后 LC51 选做）

 **套壳题：**LC17（电话号码的字母组合）——用回溯，外表是手机键盘，内核是组合枚举。

检验标准：熟练写出二叉树前中后序遍历（递归+迭代）；独立写出全排列、子集、组合、分割的回溯代码。

第六阶段A：图论体系（模板化·战术喘息）

预计时间 约4-6周

相比第五阶段的烧脑回溯，图论算法高度模板化——背模板、套模板。把它放在DP之前，让大脑在模板的节奏中恢复信心，同时巩固DFS/BFS的应用。

学什么（按顺序，非常关键）：

建图 → 图遍历(DFS/BFS) → 并查集 → Kruskal → Prim →
Dijkstra → SPFA（选学） → Floyd → 拓扑排序

为什么是这个顺序：

1. **图遍历对并查集**：先用DFS/BFS亲眼看到连通块长什么样，再用并查集用 $O(1)$ 近似管理连通块。直觉在前，工具在后。
2. **并查集对Kruskal**：Kruskal的核心就是并查集的find和union——模板直接平移，衔接最紧密。
3. **Kruskal对Prim**：同为最小生成树，Kruskal是边贪心，Prim是点贪心。放在一起直观对比。
4. **Prim对Dijkstra**：代码结构高度相似！改变一行代码就能从生成树变成最短路，这是整条路线最爽的“Aha时刻”。
5. **Dijkstra到SPFA**：从无负权到有负权的自然扩展，但SPFA仅为选学。
6. **SPFA到Floyd**：从单源到多源，从稀疏图到稠密图。

7. **拓扑排序**：独立于最短路径体系，解决“依赖顺序”问题，放在最后不影响衔接。

6.1 建图

掌握邻接表：`vector<int> G[N]`（无权图）和
`vector<pair<int,int>> G[N]`（带权图）。链式前向星选学。

6.2 图遍历

把第五阶段的DFS/BFS套在新图上，练5-10题。弄清楚什么时候深搜、什么时候广搜。

6.3 并查集

练2-3题，闭眼写出模板。掌握路径压缩和按秩合并。

6.4 最小生成树

- **Kruskal**：用刚学的并查集判环，贪心加边。练2题。
- **Prim**：贪心选点，代码骨架和Dijkstra几乎一致，为下一个算法埋伏笔。

6.5 最短路径

- **堆优化Dijkstra**（无负权单源， $O(m \log n)$ ，竞赛首选）：和Prim对比着学，找“改哪一行从生成树变成最短路”
- **SPFA（选学）**（有负权单源）：Bellman-Ford的队列优化版。**非负权图一律用 Dijkstra，SPFA 只在你明确看到题目允许负权边时才考虑。**

- **Floyd** (多源, $n \leq 200$, $O(n^3)$): 三重循环的经典
- 每种练2-3道模板题, 不要贪多

6.6 拓扑排序

Kahn算法 (入度+队列), 练3-5题。会出现“先修课程”“任务排序”等应用题。

图论学习铁律: 先背熟模板套题, 第一遍不深究证明。每学完一个算法, 必须能回答:

- ① 解决什么问题?
- ② 时间复杂度?
- ③ 限制条件 (负权、负环、有向无环)?

额外提醒: SPFA 了解即可, 日常练习默认用 Dijkstra。非负权图用 SPFA 是个坏习惯。

图论配套题目:

- 图遍历: 洛谷 P1141 (BFS)
- 并查集: 洛谷 P3367 (模板), P1111, P1196
- 最小生成树: 洛谷 P3366 (模板)
- 最短路: P3371 (Dijkstra), P4779 (堆优化Dijkstra), P3385 (SPFA选做)
- 拓扑排序: P1113, P4017

 **套壳题:** P1111 (修复公路, 并查集)、P4017 (最大食物链计数, 拓扑排序)

检验标准：熟练用邻接表建图；闭眼写并查集模板；套用Dijkstra、Floyd模板；能说出Prim和Dijkstra代码的区别。

第六阶段B：系统贪心（从图论的贪心中自然生长）

预计时间 约1-3周（打基础，建分类直觉即可）

图论里你已经见过了：Kruskal、Prim、Dijkstra的核心全是贪心——“每次做当前最优选择”。现在是时候把这种直觉系统化了。**本轮只要求能识别贪心题型、按分类套用策略，第二轮再深究。**

贪心分类刷题

类别	核心思想	配套题
排序贪心	先按某种规则排序，再顺序处理	洛谷 P1223；力扣 LC455
区间贪心	按端点排序后选择/覆盖	洛谷 P1803；力扣 LC435, LC452；洛谷 P2082
优先级贪心	每次选性价比/权值最大或最小的元素	洛谷 P1090；力扣 LC122, LC134

类别	核心思想	配套题
构造贪心	按规则逐步构造解	洛谷 P1094, P1208; 力扣 LC860, LC55

贪心与DP的鉴别（刻意制造失败）

学完贪心后，打开洛谷 P1048（01背包），尝试用贪心写——比如每次选性价比最高的物品。你会发现贪心解连样例都过不了。**这个失败经历是你进入DP的最强动机。**

判断方法：先用贪心想，举极端反例。如果反例能推翻贪心策略，或者“当前选择会影响到后续的选择空间”，那就是DP。

贪心配套题目：

- 排序贪心：P1223, LC455
- 区间贪心：P1803, LC435, LC452, P2082
- 优先级贪心：P1090, LC122, LC134
- 构造贪心：P1094, P1208, LC860, LC55

检验标准：能识别一道题是否适合贪心；能按类别套用贪心策略；亲手用贪心写01背包并看到它失败。

第七阶段：终极试炼（动态规划·一轮基础）

预计时间 约3-5周（见识核心模型，不碰高难变种）

从贪心在01背包上的失败开始你的DP之旅。蓝桥杯省赛到国赛，DP至少2-3题，是区分奖项的核心战场。但本轮的目标是**建立DP思维骨架**——理解状态定义、转移方程、记忆化与递推的关系。

7.1 记忆化搜索（绝对桥梁，不可跳过）

01背包的笨办法是什么？递归枚举每个物品选或不选——这就是回溯。回溯超时，解药是加一个memo数组，把算过的状态存起来。这就是记忆化搜索——从回溯到DP最自然的桥梁。

三步走（必须亲手写一遍）：

1. **斐波那契**：裸递归（指数级）→ 加备忘录 → 自底向上递推（DP）
2. **爬楼梯**（LC70）：同上三步，感受一维递推
3. **打家劫舍**（LC198）：一维DP里最有代表性的“选与不选”模型

亲眼看到三个版本的区别，你会顿悟：**DP的本质就是把递归的重叠子问题存起来，避免重复劳动。**

实操演示：LC329 矩阵中的最长递增路径

第一步：暴力 DFS（会超时）——从每个格子出发递归。

第二步：加 memo——用 `memo[r][c]` 存结果，改动不超过5行，时间从指数级降到 $O(mn)$ 。

第三步：体会“状态参数化”——为什么 memo 的下标是 (r,c) ？因为重叠子问题完全由 (r,c) 决定。这个思考就是推导DP状态定义的起点。

7.2 动态规划系统推进（一轮范围）

线性DP（细分为四类）：练的时候分类推进，每类做到能识别、能定义。

类型	核心思路	代表题
序列型	$dp[i]$ 表示以第 i 个元素结尾的某种最优值	LC300 (LIS) , LCS
区间/计数型	$dp[i]$ 表示前 i 个元素中的某种最优值	LC198, LC53
坐标型（网格路径）	$dp[i][j]$ 表示到达 (i,j) 的最优值	P1002, P1216
划分型	$dp[i]$ 表示将前 i 个元素分成若干段的最优值	LC72

各类型练2-3题，每做一题先判断它属于哪一类型，然后用对应的状态定义模板去套。

- **模板题（LIS/LCS）：**各写1-2题，吃透底层逻辑。
- **背包问题：**01背包、完全背包、分组背包，分清区别。每种练2-3题。强烈建议在纸上手动填一次dp数组。
- **滚动数组优化：**二维dp压成一维，解决内存超限。
- **状态压缩DP入口：**做1-2道入门题感受思路即可。

DP配套题目：

- **记忆化过渡：**LC509, LC70, LC198（三步走）；LC329（实操演示）

- DP基础：洛谷 P1216, P1002, P1020；力扣 LC53, LC300
- 背包：洛谷 P1048 (01背包) , P1616 (完全背包) , P1757 (分组背包)
- DP提高 (选做)：洛谷 P1115, P1434；力扣 LC72, LC213, LC474, LC494

 **套壳题：**LC198 (打家劫舍，线性DP)、LC494 (目标和，背包变种)

检验标准：能独立推导线性DP各类型和背包的状态转移方程；独立写出01背包和完全背包；能说出记忆化搜索和DP的关系。

每个算法必须回答的三个问题

在第三阶段至第七阶段的每个算法模块学习后，在错题本上写下这三个问题的答案：

1. **解决什么问题？** (用一句话说清这个算法的应用场景)
2. **数据范围是多少？** (这个算法能承受多大的 n ，超出就要换方案)
3. **什么时候用？**
(看到题目中的什么关键词会让你想到这个算法)

三个问题写完，这个算法才算是你的。

🚀 一轮打基础，二轮再拔高。保持节奏，享受思考。